Estimating cosmological parameters from galaxy power spectrum using `BayesFlow`

— Project Report —
Simulation Based Inference

Maria Levina, 230579

Rahul Vishwkarma, 236862

August 17, 2025

TU Dortmund University

Contents

1 Introduction

This report aims our proceedings on the project we worked on during the course "Simulated-based Inference" during summer term 2025 at the TU Dortmund university. The goal of the project was to apply theoretical concepts learned in the course of the term on a practical task and present the results. Here our task was to estimate the posterior distribution of underlying cosmological parameters using noisy observations of the galaxy power spectrum.

The structure of the universe encodes information about the history of the universe itself, and the researchers are trying to deduct the initial conditions of Big Bang and following expansion process from the current matter-energy composition of the universe (Pettini, 2015). The matter power spectrum $P(k)$ describes how matter-density fluctuations change with scale k (*spatial frequency*), and it is sensitive to several cosmological parameters. Here we tried to infer the posterior using neural posterior estimation (NPE) with learned summary statistics (Radev et al., 2020), as implemented in the BayesFlow framework for amortized Bayesian inference (Radev et al., 2023) and we only focus on parameters $\theta = (H_0, \Omega_m, n_s)$.

In the following sections we will first briefly describe the idea of the neural posterior estimation (NPE) and formulate model assumptions. Then we will assess the features of the simulated data, following by description of the framework, that was used for training and inference, and its components. And finally, the results of the estimation as well as the model diagnostics will be presented and discussed.

2 General principle of NPE

As it was already said, the goal of this project was to assess neural posterior estimation of the parameters in case of matter power spectrum and cosmological parameters. We will first introduce some notations here and then briefly describe the principle of neural posterior estimation. We denote the observations as y and underlying parameters will be summarized in a parameter vector θ. We further denote parameter prior distribution by $p(\theta)$. Hence, the posterior can be denoted by $p(\theta \mid y)$. The distribution $p(y)$ is called marginal likelihood and is usually hard to calculate because it is a difficult integral to solve. However, the Bayes' rule yields:

$$p(y)\, p(\theta \mid y) \;=\; p(\theta, y) \;=\; p(\theta)\, p(y \mid \theta).$$

Therefore, the training data $(\theta^{(s)}, y^{(s)})$ can be considered to follow following distributions (here the subscript (s) stands for *a sample*):

$$y^{(s)} \sim p(y) \quad \text{and} \quad \theta^{(s)} \sim p(\theta \mid y^{(s)}),$$

and by learning the distribution of $\theta^{(s)}$ conditioned on $y^{(s)}$ we can approximate target posterior. In the neural posterior estimation the idea is to approximate $p(\theta \mid y)$ with a generative neural network $q_\phi(\theta \mid y)$. Training dataset is needed for this network, and

for the data simulation, we will use the right part of the equation above, i.e. we sample $\theta^{(s)} \sim p(\theta)$ and then sample $y^{(s)} \sim p(y \mid \theta^{(s)})$ to get samples $(\theta^{(s)}, y^{(s)})$ from the joint distribution $p(\theta, y)$ for $s = 1, \ldots, S$ times.

3 Model assumptions and data generating process

3.1 Prior definition

The parameter vector θ combines three different parameters: Hubble constant H_0, matter density Ω_m and spectral index n_s. Here we consider the single parameters to be independent, since they are representing very different aspects of the physical system. Therefore, it holds:

$$p(\theta) = p(H_0) \cdot p(\Omega_m) \cdot p(n_s),$$

which makes the sampling from prior much easier. One can sample from the three priors separately such that for $\theta^{(s)} = (H_0^{(s)}, \Omega_m^{(s)}, n_s^{(s)})$ it holds $\theta^{(s)} \sim p(\theta)$. Here, we decided to model all three priors by uniform distributions as follows:

$$H_0 \sim \mathcal{U}(30,\, 100), \qquad \Omega_m \sim \mathcal{U}(0.1,\, 0.6), \qquad n_s \sim \mathcal{U}(0.8,\, 1.5).$$

These ranges comfortably include the true parameter estimates (see Planck et al. (2020)) while leaving room for model deviations.

3.2 Data-generating process

We generate *synthetic* observations of the matter power spectrum $P(k)$ using CAMB algorithm (Lewis et al., 2025). The wavenumber grid consists of $K = 256$ different values of $k \in [10^{-4},\, 10]$, that are evenly spaced on the logarithmic scale. For a single simulation of matter power spectrum set of values for cosmological parameters (H_0, Ω_m, n_s) has to be given. These parameters are combined in one parameter vector $\theta = (H_0, \Omega_m, n_s)$ and we therefore denote the observations $P_\theta(k)$ accordingly. The values of θ for CAMB simulation are sampled from the prior described above.

For each θ, CAMB returns observations $P_\theta(k)$ for different values of k. An example of such spectrum can be seen in the figure 1. Every time several spectra (a batch) are produced and the distribution of their values is then observed (see for example, the left panel of the figure 2). As we can see, the distribution here is highly skewed. Since all values are non-negative, we decided to use log-transformed values of $P(k)$ because their distribution is less skewed (see the right panel of the figure 2).

Additional measurement noise is added to the observed spectrum. Here we add homoscedastic Gaussian noise ε in log-space, i.e.:

$$\varepsilon \sim \mathcal{N}\big(0,\ \Sigma\big), \qquad \Sigma \ = \ 0.2\, I_K \ = \ \mathrm{diag}(\underbrace{0.2, \ldots, 0.2}_{K}),\ K = 256.$$

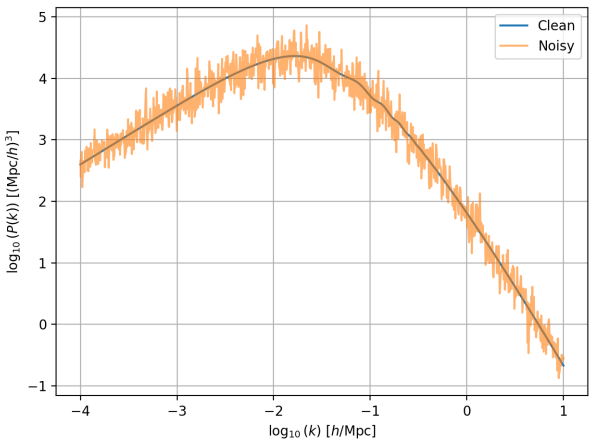

Figure 1: An example of matter power spectrum with added noise, shown on a logarithmic scale

Therefore, the observations used for neural posterior estimation y can be obtained as follows:

$$y \;=\; \log_{10} P_\theta(k) \;+\; \varepsilon.$$

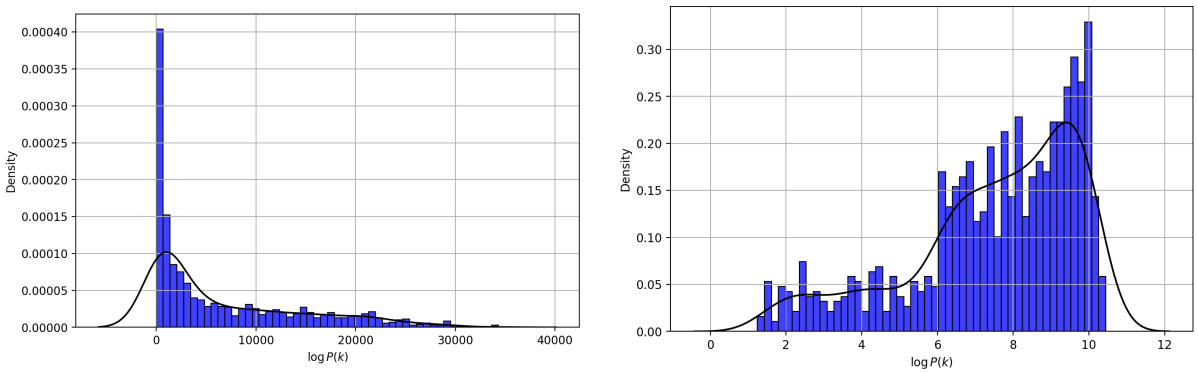

Figure 2: Distribution of $P(k)$ - histogram and KDE function of original values (left) and with logarithmic transformation applied (right)

4 Model framework

Here we describe the building blocks of NPE in terms of the `BayesFlow` framework (Radev et al., 2023). The pipeline consists of a simulator and an approximator, that combines summary and inference networks and connects them with the simulator via an adapter. The specific role of each of these part will be described accordingly. The project code was written in Python.

4.1 Simulator

In the simulator the joint model $p(\theta, y) = p(\theta)\, p(y \mid \theta)$ is defined in a way that later we can draw samples from it. It combines prior and likelihood definition and its internal procedure is already described in the previous section.

Here the uniform priors were implemented using random number generator with the given value ranges and CAMB algorithm was used to implement likelihood. We used $\Omega_b = 0.05$ (baryon density) as an additional (fixed) parameter.

4.2 Approximator

The approximator consists of: (i) a **summary network** $h(y)$ and (ii) a **conditional flow** $q_\phi(\theta \mid s(y))$ which are connected via (iii) an **adapter**.

I. Summary network: 1D CNN The summary network computes summary statistics of the simulated observations so that the input for the inference network doesn't vary in size. Here a convolutional neural network is used, since the spectrum data is not interchangeable and values depend on wavenumber k. CNN transforms the logarithmized spectrum on a fixed grid of length 256 (one channel) and outputs a 32-dimensional vector ($y \Rightarrow h(y)$):

- **Input layer:** $(128, 256, 1)$ (batch size, length, channels).
- **Layer 1:** `Conv1D`(32 filters, kernel size: 3, padding=`same`, `ReLU` activation function) $\rightarrow$ `MaxPooling1D`(pool size 2).
- **Layer 2:** `Conv1D`(64, 3, `same`, `ReLU`) $\rightarrow$ `MaxPooling1D`(pool size 2).
- **Layer 3:** `Conv1D`(32, 3, `same`, `ReLU`) $\rightarrow$ `GlobalAveragePooling1Dran`.
- **Output layer:** `Dense`(latent_dim = 32, activation=`ReLU`).

II. Inference networks The inference network $q_\phi(\theta \mid y))$ network needs to be trained with a specific loss in order to be close to the true posterior, and here the KL-divergence is used to optimize parameters of the network ϕ, using known prior distribution $p(\theta)$:

$$\begin{aligned}\hat{\phi} &= \arg\min_\phi \mathbb{E}_{p(\theta)} KL(p(\theta \mid y) \| q_\phi(\theta \mid y)) = \arg\min_\phi \mathbb{E}_{p(\theta)} \mathbb{E}_{p(\theta|y)} \log \frac{p(\theta \mid y)}{q_\phi(\theta \mid y)} \\ &= \arg\min_\phi \mathbb{E}_{p(\theta)} \mathbb{E}_{p(\theta|y)} \log(p(\theta \mid y) - q_\phi(\theta \mid y)).\end{aligned}$$

The first term in brackets is constant w.r.t. optimization term, so the whole expression can be reformulated as follows:

$$\hat{\phi} = \arg\max_\phi \mathbb{E}_{p(\theta,y)} q_\phi(\theta \mid y)).$$

We model the conditional posterior $q_\phi(\theta \mid s(y))$ for $\theta = (H_0, \Omega_m, n_s)$ using *conditional coupling flows* conditioned on the CNN summary $h(y)$. We instantiate two alternatives: affine and spline coupling flow - to compare their results later.

Affine coupling flow is usually used for low-dimensional and uni-modal posteriors. Here the form of the posterior is unknown to us, therefore spline coupling flow may deliver better results if the posterior is not uni-modal. First provides a stable baseline; the 12-layer depth and random permutations help the affine flow capture moderate curvature while remaining computationally light. And spline coupling layers may capture skewness and local nonlinearity in $q_\phi(\theta \mid s(y))$, which might yield sharper and better-calibrated posteriors than affine coupling, at a moderate extra cost.

III. Adapter The adapter brings different parts of the model together and defines which variables are the inference variables and which variables represent the summarized output of the simulator. In both the affine and spline setups we enable workflow-level standardization for both the summary variables and the inference variables. This improves numerical conditioning for both the CNN and the flow during training and inference. Unscaled numerical values may have a strong negative influence the quantity of the neural network approximation which is designed for standardized inputs. Also single parameters in the vector θ have completely different scales, which will affect the quality of the model negatively if we don't standardize them (see BayesFlow user guide).

5 Training

We generate the training and validation datasets separately and using the simulator, ensuring no overlap and no information leakage. Training set consists of $10,000$ samples; validation set consists of $1,000$ samples. The model is trained in offline mode because the CAMB algorithm can be quite slow in the online training mode.

The model was trained over max. 200 epochs with 200 batches per epoch and 128 samples per batch. This yields $200 \times 200 = 40{,}000$ gradient updates and $200 \times 200 \times 128 = 5.12 \times 10^6$ sample-passes in total. Per epoch, 25600 examples are drawn (with replacement) from the training dataset.

At the end of every epoch, the model is evaluated on the validation set to track negative log-likelihood (NLL) and to detect overfitting.

We use the default BayesFlow training settings throughout. No custom optimizer hyperparameters, learning–rate schedules, gradient clipping, weight decay, or callbacks are specified beyond the library defaults (typically Adam with its standard parameters). Both affine and spline workflows use the same unchanged defaults for a fair comparison.

6 Model diagnostics

We assess optimization progress and posterior estimation quality using default diagnostics in BayesFlow package on the test set ($N = 1,000$). The toolkit produces: (i) convergence curves (training/validation NLL), (ii) simulation–based calibration (SBC) via calibration

ECDF difference, and (iii) parameter recovery plots. We report each below, together with the expected behavior under a well-calibrated model.

6.1 Convergence

Training proceeds smoothly, with steadily decreasing loss and only minor stochastic fluctuations for both models. After approximately 125 epochs, the training and validation curves track each other almost perfectly, with a negligible gap — indicating stable optimization and minimal generalization error.

Final loss for the model with affine flow coupling as inference network reaches -2.3223 on train data and -2.3403 on validation set. The loss values are negative since they are presented on the log-scale (see figures 3). The validation loss is slightly more negative than the training loss and indicates no overfitting. In case of spline coupling used for inference network training and validation losses decrease down to -3.0545 and -3.0140 respectively. Because smaller NLL indicates a better fit, the spline model outperforms the affine baseline.

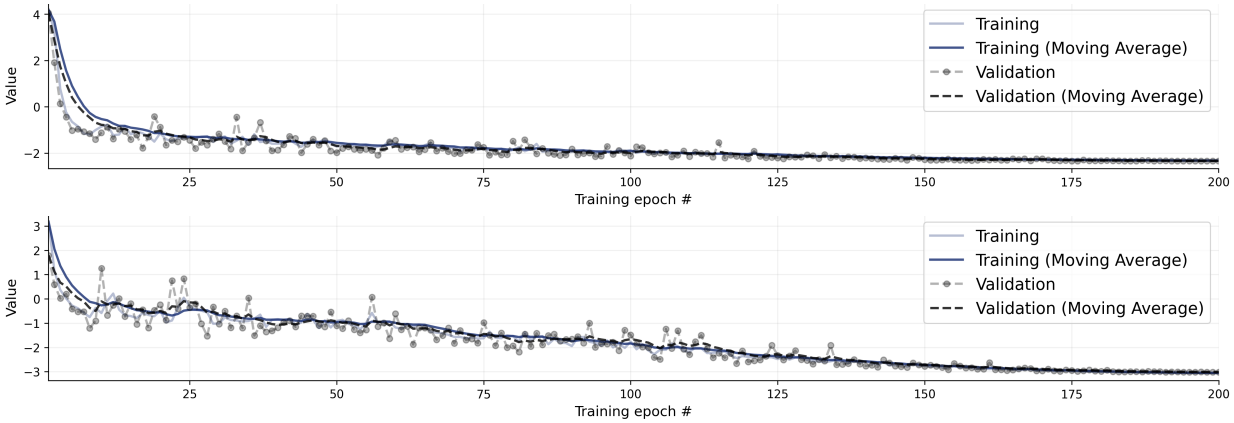

Figure 3: Loss trajectories of models with affine (top panel) and spline (bottom panel) coupling flows as inference network.

6.2 Approximator Diagnostics

We evaluate our BayesFlow posterior approximator using two diagnostic categories: (1) *computational faithfulness through SBC*, which assesses whether the approximate posterior matches the true posterior, and (2) *model sensitivity through parameter recovery*, which evaluates how informative the data are about the parameters of interest.

Calibration ECDF Difference We evaluate calibration using the ECDF difference (empirical CDF minus the diagonal). For a well–calibrated posterior, the curve should hover around zero and remain within the Monte Carlo 95% pointwise bands.

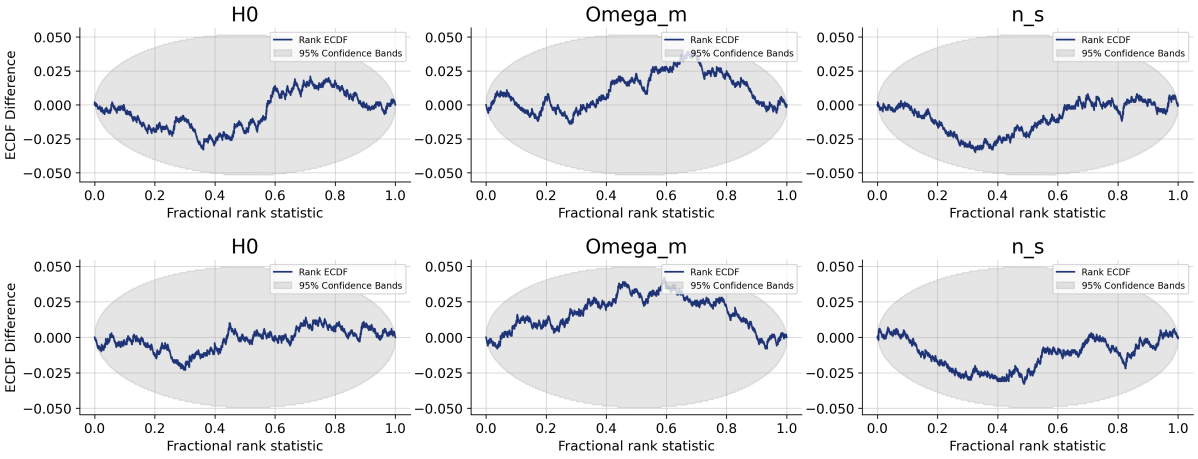

Figure 4: ECDF difference for affine (top) and spline (bottom) coupling flow models.

- H_0: For both models, the ECDF–difference curve remains tightly centered around zero, indicating good calibration.
- Ω_m: For the spline model, the ECDF–difference that the posterior mean consistently lies above the true value—i.e., a small systematic overestimation of Ω_m. This bias is however modest and remains within the 95% calibration band;
- n_s: Both models show only minute deviations from zero, with a very slight U–shaped pattern across some quantiles. This suggests a tiny underestimation of n_s (posterior means marginally below the true value), but the effect is small and remains within the 95% confidence band;

For both the **affine** and **spline** models, the ECDF–difference curves for all three parameters lies within the 95% bands, indicating no statistically significant miscalibration (see figure 4); any structure observed is minor and within expected sampling variability.

Parameter recovery To evaluate the accuracy and calibration of the posterior estimates, parameter recovery plots were generated for the affine coupling model and the spline coupling model across the three cosmological parameters (H_0, Ω_m, n_s). These plots compare the true simulated parameter values against the corresponding posterior means, with alignment along the 45° diagonal line indicating unbiased recovery.

For the affine coupling flow model, the recovery results reveal greater variability and less precise calibration compared to the spline-based approach. Across all three parameters, the posterior estimates are widely scattered around the diagonal, suggesting higher uncertainty in the inference process. For H_0, several estimates fall systematically below the diagonal at higher true values, indicating a tendency toward underestimation in this case ($r = 0.894$, Pearson's correlation coefficient). For Ω_m, the a trend with a slope less than one can be seen and exhibits the largest variance among the three parameters, reflecting weaker recovery and reduced sensitivity ($r = 0.766$). For n_s, the estimates are

more closely aligned with the diagonal, however some degree of variance persists, and the slope is slightly shallower than one ($r = 0.882$).

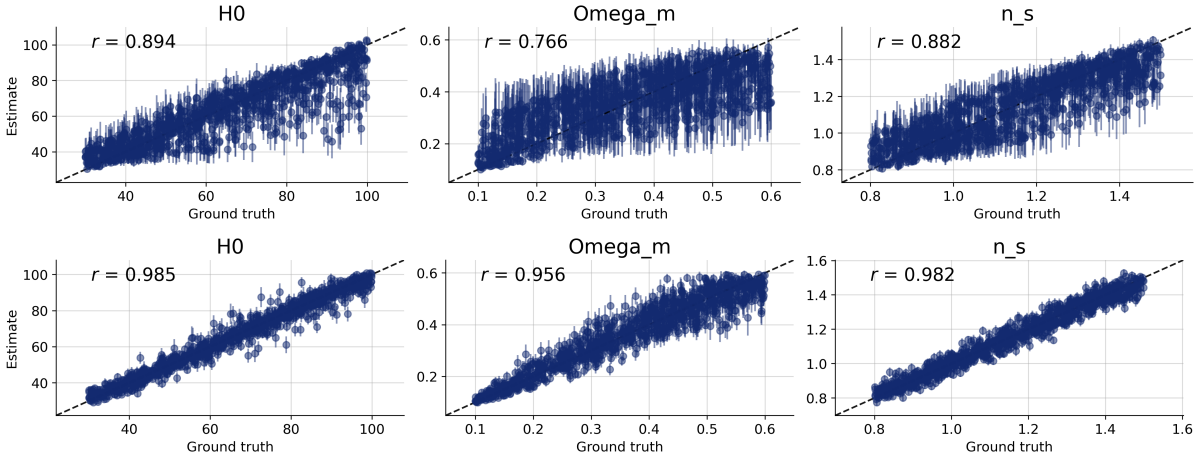

Figure 5: Parameter recovery plots for single parameters for affine (top) and spline (bottom) coupling flow models.

On the contrary, the spline model demonstrates substantially improved recovery performance. For both H_0 ($r = 0.985$) and n_s ($r = 0.982$), the posterior estimates are well-aligned with the diagonal and exhibit minimal variance, indicating highly accurate and unbiased recovery. This highlights the superior representational capacity of the spline-based normalizing flow. For Ω_m, while the estimates also align closely with the diagonal ($r = 0.956$), some scatter remains, particularly at higher parameter values, suggesting slightly reduced recovery performance relative to the other two parameters. Nevertheless, overall recovery quality for the spline model is significantly higher than that of the affine model.

In summary, both models converge well during the training and show satisfactory decrease in loss. However, the spline coupling model provides more faithful and robust parameter recovery across the cosmological parameters, with particularly strong results for H_0 and n_s. The affine model, while capable of capturing general trends, suffers from higher variance and systematic biases, especially in the recovery of Ω_m.

7 Discussion & Conclusion

In this report we summarized the results of our project work on neural posterior estimation. Our goal was to infer the joint posterior distribution of three cosmological parameters using simulated noisy observations of the matter power spectrum. We described the general principle of NPE method and the construction of our experiment using `BayesFlow` framework. We then assessed the quality of two different approaches (affine vs. spline coupling flow as inference networks) using package built-in model diagnostics.

As a result, both model tend to converge well and seem to be quite well calibrated. As for the single parameter estimates, the diagnostics show that Ω_m may be slightly overestimated by both models, its bias as well as the bias for other two parameters (Hubble constant H_0 and spectral index n_s) estimation is not significant. In general, the diagnostics also indicate that spline couplings achieve tighter, well-calibrated posteriors compared to affine couplings. These findings underscore the importance of flexible posterior approximators, such as spline-based coupling flows, for accurate simulation-based inference in cosmology.

References

Pettini, M. (2015). Introduction to cosmology. *Lecture*, *14*.

Planck, C., Aghanim, N., Akrami, Y., Ashdown, M., Aumont, J., Baccigalupi, C., Ballardini, M., Banday, A. J., Barreiro, R. B., Bartolo, N., et al. (2020). Planck 2018 results. vi. cosmological parameters [arXiv:1807.06209]. *Astronomy & Astrophysics*, *641*, A6. https://doi.org/10.1051/0004-6361/201833910

Radev, S. T., Mertens, U. K., Voss, A., Ardizzone, L., & Köthe, U. (2020). BayesFlow: Learning complex stochastic models with invertible neural networks. *IEEE transactions on neural networks and learning systems*, *33*(4), 1452–1466.

Radev, S. T., Schmitt, M., Schumacher, L., Elsemüller, L., Pratz, V., Schälte, Y., Köthe, U., & Bürkner, P.-C. (2023). BayesFlow: Amortized Bayesian workflows with neural networks. *Journal of Open Source Software*, *8*(89), 5702.

Lewis, A., Challinor, A., & Lasenby, A. (2025). *CAMB: Code for anisotropies in the microwave background* [Accessed: 2025-08-16].